

Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....	6
1.1 Описание входных данных.....	8
1.2 Описание выходных данных.....	9
2 МЕТОД РЕШЕНИЯ.....	11
3 ОПИСАНИЕ АЛГОРИТМОВ.....	15
3.1 Алгоритм метода setReady класса cl_base.....	15
3.2 Алгоритм метода printTree класса cl_base.....	16
3.3 Алгоритм метода printTreeReady класса cl_base.....	17
3.4 Алгоритм метода findObjectFromBranch класса cl_base.....	18
3.5 Алгоритм метода findObjectFromRoot класса cl_base.....	19
3.6 Алгоритм конструктора класса cl_application.....	19
3.7 Алгоритм метода build_tree_objects класса cl_application.....	20
3.8 Алгоритм метода exec_app класса cl_application.....	21
3.9 Алгоритм конструктора класса cl_2.....	22
3.10 Алгоритм конструктора класса cl_3.....	22
3.11 Алгоритм конструктора класса cl_4.....	23
3.12 Алгоритм конструктора класса cl_5.....	23
3.13 Алгоритм конструктора класса cl_6.....	24
3.14 Алгоритм функции main.....	24
3.15 Алгоритм конструктора класса cl_base.....	24
3.16 Алгоритм метода get_subordinate_by_index класса cl_base.....	25
3.17 Алгоритм метода get_subordinate_by_name класса cl_base.....	26
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....	27
5 КОД ПРОГРАММЫ.....	43
5.1 Файл cl_2.cpp.....	43
5.2 Файл cl_2.h.....	43

5.3 Файл cl_3.cpp.....	44
5.4 Файл cl_3.h.....	44
5.5 Файл cl_4.cpp.....	44
5.6 Файл cl_4.h.....	45
5.7 Файл cl_5.cpp.....	45
5.8 Файл cl_5.h.....	45
5.9 Файл cl_6.cpp.....	46
5.10 Файл cl_6.h.....	46
5.11 Файл cl_application.cpp.....	46
5.12 Файл cl_application.h.....	48
5.13 Файл cl_base.cpp.....	49
5.14 Файл cl_base.h.....	52
5.15 Файл main.cpp.....	53
6 ТЕСТИРОВАНИЕ.....	55
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	57

1 ПОСТАНОВКА ЗАДАЧИ

Первоначальная сборка системы (дерева иерархии объектов, модели системы) осуществляется исходя из входных данных. Данные вводятся построчно. Первая строка содержит имя корневого объекта (объект приложение). Номер класса корневого объекта 1. Далее, каждая строка входных данных определяет очередной объект, задает его характеристики и расположение на дереве иерархии. Структура данных в строке:

«Наименование головного объекта» «Наименование очередного объекта» «Номер класса принадлежности очередного объекта»

Ввод иерархического дерева завершается, если наименование головного объекта равно «endtree» (в данной строке ввода больше ничего не указывается).

Поиск головного объекта выполняется от последнего созданного объекта. Первоначально последним созданным объектом считается корневой объект. Если для головного объекта обнаруживается дуближ имени в непосредственно подчиненных объектах, то объект не создается. Если обнаруживается дуближ имени на дереве иерархии объектов, то объект не создается. Если номер класса объекта задан некорректно, то объект не создается.

Вывод иерархического дерева объектов на консоль.

Внутренняя архитектура (вид иерархического дерева объектов) в большинстве реализованных моделях систем динамически меняется в процессе отработки алгоритма. Вывод текущего дерева объектов является важной задачей, существенно помогая разработчику, особенно на этапе тестирования и отладки программы.

В данной задаче подразумевается, что наименования объектов уникальны. Система содержит объекты пяти классов, не считая корневого. Номера классов: 2,3,4,5,6.

Расширить функциональность базового класса:

- метод поиска объекта на ветке дерева иерархии от текущего по имени (метод возвращает адрес на найденный объект или значение nullptr). Передается один параметр строкового типа, содержит наименование искомого объекта. Если на искомой ветке дерева иерархии наименование объекта не уникально или отсутствует, то возвращает значение nullptr;

- метод поиска объекта на дереве иерархии по имени (метод возвращает адрес на найденный объект или значение nullptr). Передается один параметр строкового типа, содержит наименование искомого объекта. Если на дереве иерархии наименование объекта не уникально или отсутствует, то возвращает значение nullptr;

- метод вывода иерархии объектов (дерева или ветки) от текущего объекта (допускается использовать один целочисленный параметр со значением по-умолчанию);

- метод вывода иерархии объектов (дерева или ветки) и отметок их готовности от текущего объекта (допускается использовать один целочисленный параметр со значением по-умолчанию);

- метод установки готовности объекта, в качестве параметра передается переменная целого типа, содержит номер состояния.

Устаревший метод вывода из задачи KB_1 убрать.

Готовность для каждого объекта устанавливается индивидуально. Готовность задается посредством любого отличного от нуля целого числового значения, которое присваивается свойству состояния объекта. Изменение состояния готовности возможно, если все объекты вверх по иерархии до корневого включены, иначе установка готовности игнорируется. При отключении головного, отключаются все объекты от него по иерархии вниз по ветке. Свойству состояния объекта присваивается значение нуль.

Разработать программу:

1. Построить дерево объектов системы (в методе корневого объекта построения исходного дерева объектов).

2. В методе корневого объекта запуска моделируемой системы реализовать: 2.1. Вывод на консоль иерархического дерева объектов в следующем виде:

```
root
  ob_1
    ob_2
  ob_3
    ob_4
      ob_5
    ob_6
      ob_7
```

где: root - наименование корневого объекта (приложения).

2.2. Переключение готовности объектов согласно входным данным (командам). 2.3. Вывод на консоль иерархического дерева объектов и отметок их готовности в следующем виде:

```
root  is ready
  ob_1  is ready
    ob_2  is ready
  ob_3  is ready
    ob_4 is not ready
      ob_5 is not ready
    ob_6 is ready
      ob_7 is not ready
```

1.1 Описание входных данных

Множество объектов, их характеристики и расположение на дереве иерархии. Последовательность ввода организовано так, что головной объект для очередного вводимого объекта уже присутствует на дереве иерархии объектов.

Первая строка:

«Наименование корневого объекта»

Со второй строки:

```
«Наименование головного объекта» «Наименование очередного объекта» «Номер
класса принадлежности очередного объекта»
. . . . .
endtree
```

Со следующей строки вводятся команды включения или отключения объектов

```
«Наименование объекта» «Номер состояния объекта»
```

Пример ввода:

```
app_root
app_root object_01 3
app_root object_02 2
object_02 object_04 3
object_02 object_05 5
object_01 object_07 2
endtree
app_root 1
object_07 3
object_01 1
object_02 -2
object_04 1
```

1.2 Описание выходных данных

Вывести иерархию объектов в следующем виде:

```
Object tree
«Наименование корневого объекта»
  «Наименование объекта 1»
    «Наименование объекта 2»
      «Наименование объекта 3»
. . . . .
The tree of objects and their readiness
«Наименование корневого объекта» «Отметка готовности»
  «Наименование объекта 1» «Отметка готовности»
    «Наименование объекта 2» «Отметка готовности»
      «Наименование объекта 3» «Отметка готовности»
. . . . .
«Отметка готовности» - равно «is ready» или «is not ready»
```

Отступ каждого уровня иерархии 4 позиции.

Пример вывода:

```
Object tree
app_root
  object_01
    object_07
```

```
    object_02
      object_04
      object_05
The tree of objects and their readiness
app_root is ready
  object_01 is ready
    object_07 is not ready
  object_02 is ready
    object_04 is ready
    object_05 is not ready
```


2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект `cin` класса `istream` предназначен для ввода с клавиатуры;
- объект `cout` класса `ostream` предназначен для вывода на экран;
- объект `ob_cl_application` класса `cl_application` предназначен для объявления объекта;
- `for` - оператор цикла со счетчиком;
- `if...else` - условный оператор;
- `while` - оператор цикла с предусловием.

Класс `cl_base`:

- свойства/поля:
 - поле имя объекта:
 - наименование — `name`;
 - тип — `string`;
 - модификатор доступа — `protected`;
 - поле указатель головного объекта:
 - наименование — `p_head_object`;
 - тип — `cl_base*`;
 - модификатор доступа — `protected`;
 - поле динамический массив адресов объектов, подчиненных текущему объекту в дереве иерархии:
 - наименование — `v_child_objects`;
 - тип — `vector<cl_base>`;
 - модификатор доступа — `protected`;
 - поле готовность:
 - наименование — `is_ready`;

- тип — int;
- модификатор доступа — protected;
- функционал:
 - о метод setReady — метод установки готовности объекта;
 - о метод printTree — метод вывода иерархии объектов (дерева или ветки) от текущего объекта;
 - о метод printTreeReady — метод вывода иерархии объектов и их готовности;
 - о метод findObjectFromBranch — метод поиска объекта на ветке от текущего по имени;
 - о метод findObjectFromRoot — метод поиска объекта на дереве иерархии по имени;
 - о метод cl_base — конструктор с параметрами;
 - о метод get_subordinate_by_index — метод для получения адреса непосредственного подчиненного объекта по его индексу;
 - о метод get_subordinate_by_name — метод для получения адреса непосредственного подчиненного объекта по его имени.

Класс cl_application:

- функционал:
 - о метод cl_application — конструктор с параметром;
 - о метод build_tree_objects — метод построения исходного дерева иерархии объектов;
 - о метод exes_app — метод запуска приложения.

Класс cl_2:

- функционал:
 - о метод cl_2 — конструктор с параметрами.

Класс cl_3:

- функционал:
 - о метод cl_3 — конструктор с параметрами.

Класс cl_4:

- функционал:
 - о метод cl_4 — конструктор с параметрами.

Класс cl_5:

- функционал:
 - о метод cl_5 — конструктор с параметрами.

Класс cl_6:

- функционал:
 - о метод cl_6 — конструктор с параметрами.

Таблица 1 – Иерархия наследования классов

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
1	cl_base			базовый класс	
		cl_application	public		2
		cl_2	public		3
		cl_3	public		4
		cl_4	public		5
		cl_5	public		6
		cl_6	public		7
2	cl_application				
3	cl_2			первый производный класс	
4	cl_3			второй производный класс	
5	cl_4			третий производный класс	

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
6	cl_5			четвертый производный класс	
7	cl_6			пятый производный класс	

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм метода `setReady` класса `cl_base`

Функционал: метод установки готовности объекта.

Параметры: `int state` - состояние готовности.

Возвращаемое значение: `void` - ничего не возвращает.

Алгоритм метода представлен в таблице 2.

Таблица 2 – Алгоритм метода `setReady` класса `cl_base`

№	Предикат	Действия	№ перехода
1	state равно 0	установка защищенного свойства <code>is_ready</code> равным 0	2
			5
2		указатель <code>child</code> настраиваемый на 1-ый объект в <code>v_child_objects</code> настоящего объекта	3
3	<code>child</code> указывает на окончание <code>v_child_objects</code> настоящего объекта	вызов метода <code>set_ready</code> с параметром 0 для каждого <code>child</code>	4
		завершение метода	Ø
4		указатель <code>child</code> настраиваемый на следующий элемент в <code>v_child_objects</code> настоящего объекта	3
5		инициализация индекса <code>parent</code> на объект класса <code>cl_base</code> и присвоение ему значения защищенного свойства <code>p_head_object</code>	6
6	<code>parent</code> не равен <code>nullptr</code>		7

№	Предикат	Действия	№ перехода
			9
7	свойство is_ready объекта parent равно 0	завершение метода	∅
			8
8		присвоение указателю parent значения parent->p_head_object	6
9		присвоение защищенному свойству is_ready значения state	∅

3.2 Алгоритм метода printTree класса cl_base

Функционал: метод вывода иерархии объектов.

Параметры: int level - уровень глубины.

Возвращаемое значение: void - ничего не возвращает.

Алгоритм метода представлен в таблице 3.

Таблица 3 – Алгоритм метода printTree класса cl_base

№	Предикат	Действия	№ перехода
1		целый индекс i = 0	2
2	i меньше параметра level	вывод на экран " "	3
		вывод на экран this->name	4
3		инкремент i++	2
4		указатель child настраиваемый на 1-ый объект в v_child_objects настоящего объекта	5
5	child указывает на окончание v_child_objects настоящего объекта	вызов метода print_tree с параметром level + 1 для каждого child	6
			∅
6		указатель child настраиваемый на следующий	5

№	Предикат	Действия	№ перехода
		элемент в v_child_objects настоящего объекта	

3.3 Алгоритм метода printTreeReady класса cl_base

Функционал: метод вывода иерархии объектов и их готовности.

Параметры: int level - уровень глубины.

Возвращаемое значение: void - ничего не возвращает.

Алгоритм метода представлен в таблице 4.

Таблица 4 – Алгоритм метода printTreeReady класса cl_base

№	Предикат	Действия	№ перехода
1		целый индекс i = 0	2
2	i меньше параметра level	вывод на экран " "	3
			4
3		инкремент i++	2
4		вывод на экран name	5
5	is_ready не равно 0	вывод на экран " is ready"	6
		вывод на экран " is not ready"	6
6		указатель child настраиваемый на 1-ый объект в v_child_objects настоящего объекта	7
7	child указывает на окончание v_child_objects настоящего объекта	вызов метода print_tree_ready с параметром level + 1 для каждого child	8
			∅
8		указатель child настраиваемый на следующий элемент в v_child_objects настоящего объекта	7

3.4 Алгоритм метода findObjectFromBranch класса cl_base

Функционал: метод поиска объекта на дереве иерархии от текущего по имени.

Параметры: string name - имя объекта.

Возвращаемое значение: cl_base* - адрес объекта.

Алгоритм метода представлен в таблице 5.

Таблица 5 – Алгоритм метода findObjectFromBranch класса cl_base

№	Предикат	Действия	№ перехода
1		инициализация указателя result на объект класса cl_base и присвоение ему значения nullptr	2
2		инициализация переменной count типа int со значением 0	3
3		инициализация объекта q класса queue, содержащего в себе индексы на объекты класса cl_base	4
4		добавление в очередь q адреса настоящего объекта	5
5	q не пустое	инициализация индекса current на объект класса cl_base и присвоение ему значения первого элемента очереди q	6
			12
6		удаление первого элемента очереди	7
7	свойство name объекта current равно name	присвоение указателю result значения указателя current	8
			9
8		увеличение count на 1	9
9		указатель child настраиваемый на 1-ый объект в v_child_objects объекта current	10
10	child указывает на окончание v_child_objects объекта	добавление в очередь q указателя child	11

№	Предикат	Действия	№ перехода
	current		
			5
11		указатель child настраиваемый на следующий элемент в v_child_objects объекта current	10
12	count равно 1	возвращение указателя result	∅
		возвращение nullptr	∅

3.5 Алгоритм метода findObjectFromRoot класса cl_base

Функционал: метод поиска объекта на дереве иерархии по имени.

Параметры: string name - имя объекта.

Возвращаемое значение: cl_base* - адрес объекта.

Алгоритм метода представлен в таблице 6.

Таблица 6 – Алгоритм метода findObjectFromRoot класса cl_base

№	Предикат	Действия	№ перехода
1		инициализация указателя root на объект класса cl_base и присвоение ему значения this	2
2	root->p_head_object не равен nullptr	присвоение указателю root значения p_head_object	2
		возвращение результата вызова метода findObjectFromBranch с параметром name от объекта root	∅

3.6 Алгоритм конструктора класса cl_application

Функционал: конструктор с параметром.

Параметры: cl_base* p_head_object - адрес головного объекта.

Алгоритм конструктора представлен в таблице 7.

Таблица 7 – Алгоритм конструктора класса *cl_application*

№	Предикат	Действия	№ перехода
1		вызов конструктора базового класса <i>cl_base</i> с параметром <i>p_head_object</i>	Ø

3.7 Алгоритм метода *build_tree_objects* класса *cl_application*

Функционал: метод построения исходного дерева иерархии объектов.

Параметры: нет.

Возвращаемое значение: *void* - ничего не возвращает.

Алгоритм метода представлен в таблице 8.

Таблица 8 – Алгоритм метода *build_tree_objects* класса *cl_application*

№	Предикат	Действия	№ перехода
1		инициализация переменной <i>root_name</i> , <i>parent_name</i> , <i>child_name</i> типа <i>string</i>	2
2		ввод с клавиатуры переменной <i>root_name</i>	3
3		вызов метода <i>set_name</i> с параметром <i>root_name</i>	4
4		инициализация указателя <i>current</i> на объект класса <i>cl_base</i> и присвоение значения <i>this</i>	5
5		инициализация переменной <i>class_affil</i> типа <i>int</i>	6
6	ввод с клавиатуры <i>parent_name</i>	остановка цикла	Ø
		ввод с клавиатуры переменных <i>child_name</i> , <i>class_affil</i>	7
7	<i>class_affil</i> меньше 2 или больше 6		6
		инициализация указателя <i>parent</i> на объект класса <i>cl_base</i> и присвоение значения результата вызова метода <i>findObjectFromRoot ()</i> с параметром	8

№	Предикат	Действия	№ перехода
		parent_name от настоящего объекта	
8	parent равен nullptr		6
			9
9	результат вызова метода findObjectFromRoot() с параметром child_name от настоящего объекта не равен nullptr		6
			10
10		при значениях переменной class_affil от 2 до 6 создать объект класса cl_2, cl_3, cl_4, cl_5, cl_6 с параметрами parent и child_name	∅

3.8 Алгоритм метода exec_app класса cl_application

Функционал: метод запуска приложения.

Параметры: нет.

Возвращаемое значение: int - индикатор корректности завершения программы.

Алгоритм метода представлен в таблице 9.

Таблица 9 – Алгоритм метода exec_app класса cl_application

№	Предикат	Действия	№ перехода
1		вывод на экран "Object tree\n"	2
2		вызов метода printTree()	3
3		вывод на экран "The tree of objects and their readiness\n"	4
4		инициализация переменной name типа string	5
5		инициализация переменной state типа int	6

№	Предикат	Действия	№ перехода
6	ввод с клавиатуры name и state	инициализация указателя obj на объект класса cl_base и присвоение ему результата метода findObjectFromRoot() с параметром name	7
			8
7	obj не равен nullptr	вызов метода setReady с параметром state от указателя obj	6
			6
8		вызов метода printTreeReady	9
9		возвращение 0	∅

3.9 Алгоритм конструктора класса cl_2

Функционал: конструктор с параметрами.

Параметры: cl_base* p_head_object - указатель головного объекта, string name - имя объекта,.

Алгоритм конструктора представлен в таблице 10.

Таблица 10 – Алгоритм конструктора класса cl_2

№	Предикат	Действия	№ перехода
1		вызов конструктора базового класса cl_base с параметрами p_head_object и name	∅

3.10 Алгоритм конструктора класса cl_3

Функционал: конструктор с параметрами.

Параметры: cl_base* p_head_object - указатель головного объекта, string name - имя объекта.

Алгоритм конструктора представлен в таблице 11.

Таблица 11 – Алгоритм конструктора класса cl_3

№	Предикат	Действия	№ перехода
1		вызов конструктора базового класса cl_base с параметрами p_head_object и name	Ø

3.11 Алгоритм конструктора класса cl_4

Функционал: конструктор с параметрами.

Параметры: cl_base* p_head_object - указатель головного объекта, string name - имя объекта.

Алгоритм конструктора представлен в таблице 12.

Таблица 12 – Алгоритм конструктора класса cl_4

№	Предикат	Действия	№ перехода
1		вызов конструктора базового класса cl_base с параметрами p_head_object и name	Ø

3.12 Алгоритм конструктора класса cl_5

Функционал: конструктор с параметрами.

Параметры: cl_base* p_head_object - указатель головного объекта, string name - имя объекта.

Алгоритм конструктора представлен в таблице 13.

Таблица 13 – Алгоритм конструктора класса cl_5

№	Предикат	Действия	№ перехода
1		вызов конструктора базового класса cl_base с параметрами p_head_object и name	Ø

3.13 Алгоритм конструктора класса cl_6

Функционал: конструктор с параметрами.

Параметры: cl_base* p_head_object - указатель головного объекта, string name - имя объекта.

Алгоритм конструктора представлен в таблице 14.

Таблица 14 – Алгоритм конструктора класса cl_6

№	Предикат	Действия	№ перехода
1		вызов конструктора базового класса cl_base с параметрами p_head_object и name	Ø

3.14 Алгоритм функции main

Функционал: основной алгоритм программы.

Параметры: нет.

Возвращаемое значение: int - индикатор корректности завершения программы.

Алгоритм функции представлен в таблице 15.

Таблица 15 – Алгоритм функции main

№	Предикат	Действия	№ перехода
1		инициализация объекта ob_cl_application класса cl_application	2
2		вызов метода build_tree_objects от объекта ob_cl_application	3
3		возвращение результата вызова метода exes_app от объекта ob_cl_application	Ø

3.15 Алгоритм конструктора класса cl_base

Функционал: конструктор с параметрами.

Параметры: cl_base* p_head_object - указатель головного объекта, string name - имя объекта.

Алгоритм конструктора представлен в таблице 16.

Таблица 16 – Алгоритм конструктора класса cl_base

№	Предикат	Действия	№ перехода
1		присвоение защищенному свойству p_head_object значения параметра p_head_object	2
2		присвоение защищенному свойству name значения параметра name	3
3	p_head_object != nullptr	добавление адреса настоящего объекта в массив v_child_objects головного объекта p_head_object	∅
			∅

3.16 Алгоритм метода get_subordinate_by_index класса cl_base

Функционал: метод для получения адреса непосредственного подчиненного объекта по его индексу.

Параметры: int index - индекс.

Возвращаемое значение: cl_base* - адрес объекта.

Алгоритм метода представлен в таблице 17.

Таблица 17 – Алгоритм метода get_subordinate_by_index класса cl_base

№	Предикат	Действия	№ перехода
1	index < 1 index > (int)v_child_objects.size()	возвращение nullptr	∅
		возвращение v_child_objects с индексом index - 1	∅

3.17 Алгоритм метода `get_subordinate_by_name` класса `cl_base`

Функционал: метод для получения адреса непосредственного подчиненного объекта по его имени.

Параметры: `string name` - имя объекта.

Возвращаемое значение: `cl_base*` - адрес объекта.

Алгоритм метода представлен в таблице 18.

Таблица 18 – Алгоритм метода `get_subordinate_by_name` класса `cl_base`

№	Предикат	Действия	№ перехода
1	<code>i < v_child_objects.size()</code>		2
			4
2	<code>v_child_objects[i]->name == name</code>	возвращение <code>v_child_objects</code> с индексом <code>i</code>	Ø
			3
3		инкремент <code>i++</code>	1
4		возвращение <code>nullptr</code>	Ø

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-16.

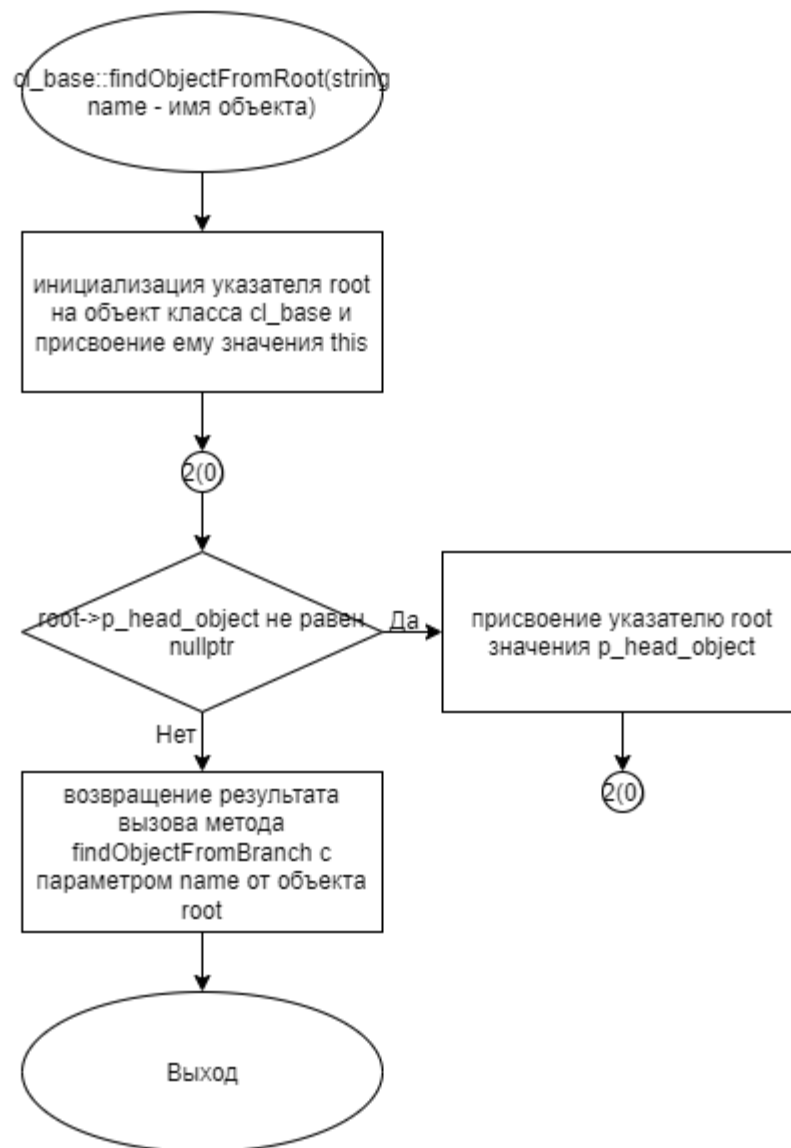


Рисунок 1 – Блок-схема алгоритма

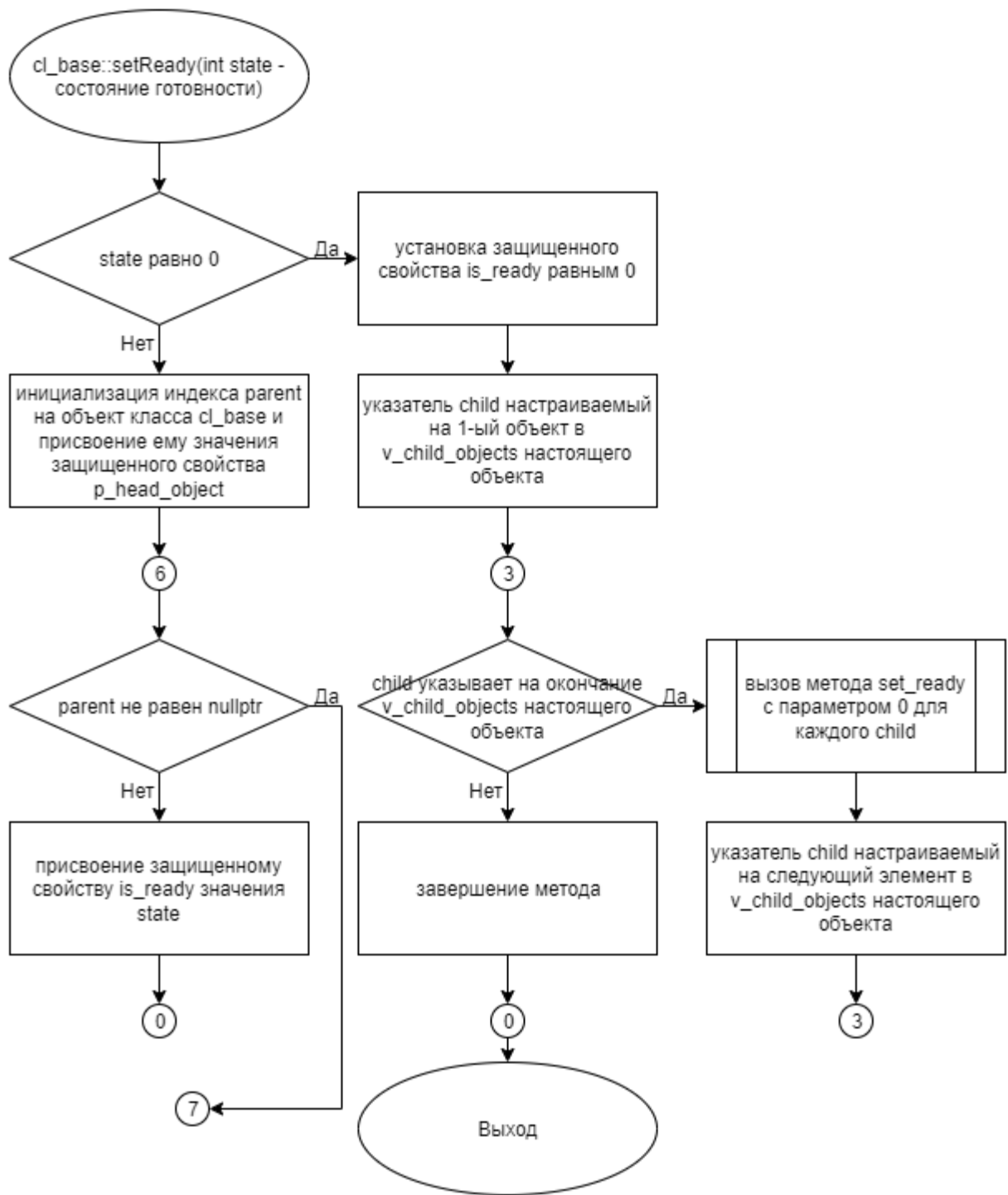


Рисунок 2 – Блок-схема алгоритма

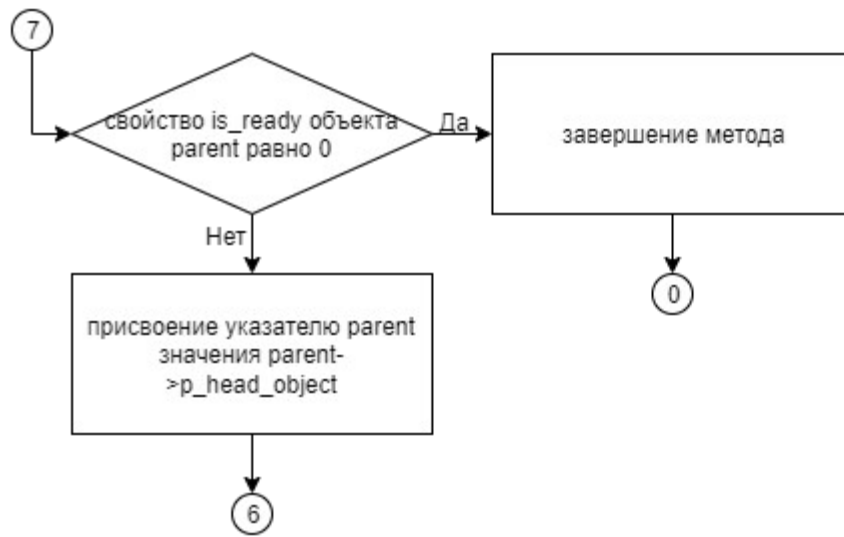


Рисунок 3 – Блок-схема алгоритма

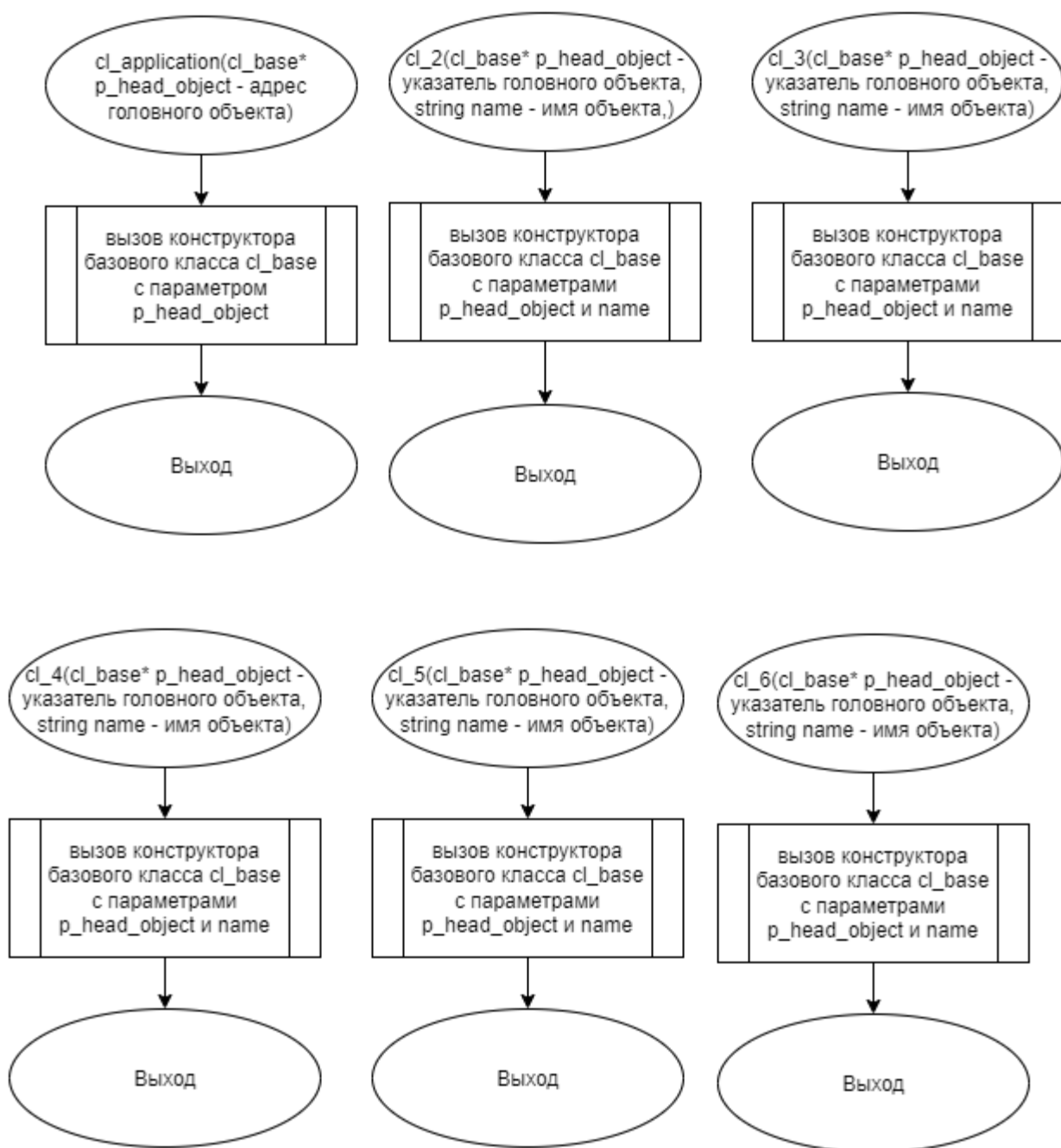


Рисунок 4 – Блок-схема алгоритма

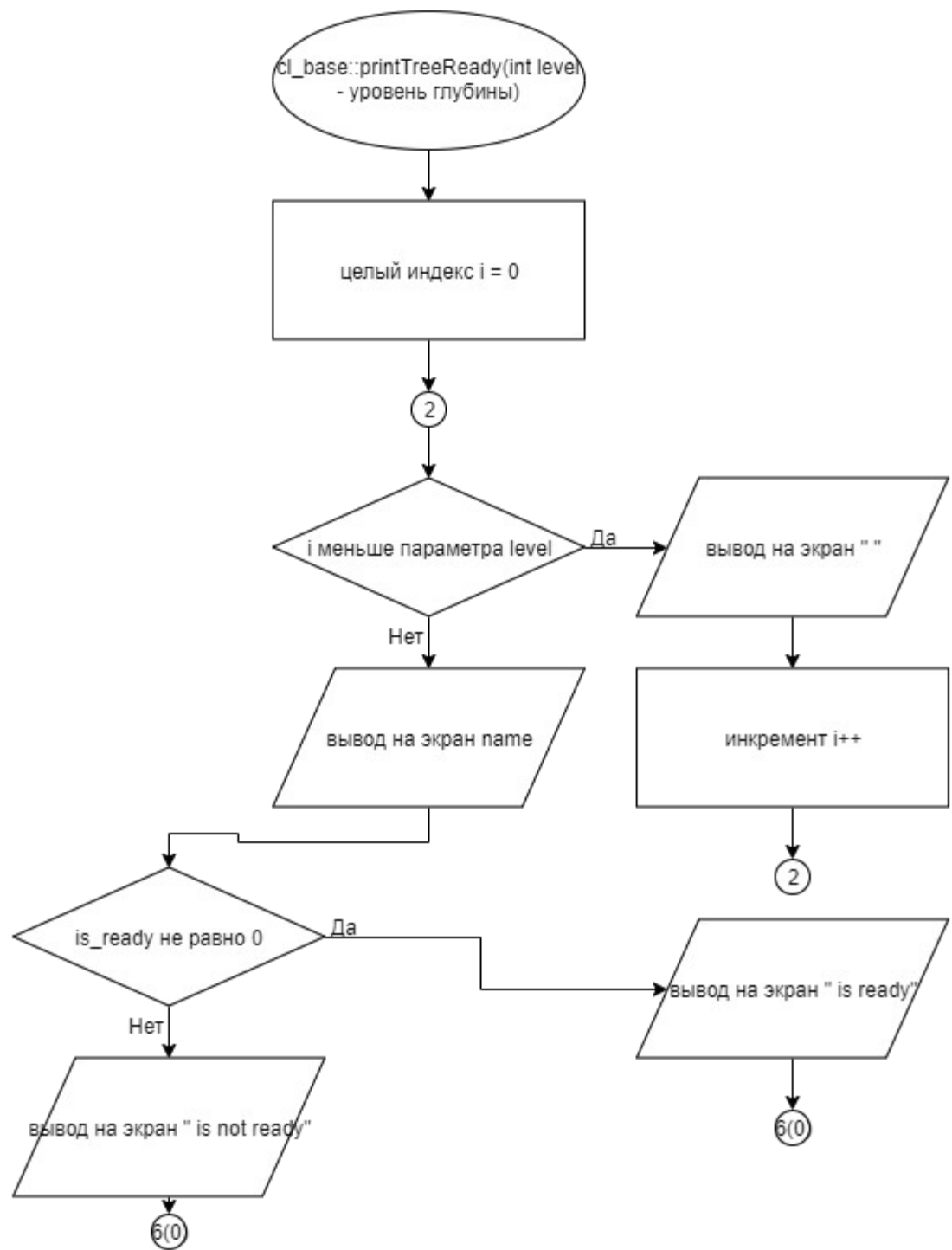


Рисунок 5 – Блок-схема алгоритма



Рисунок 6 – Блок-схема алгоритма

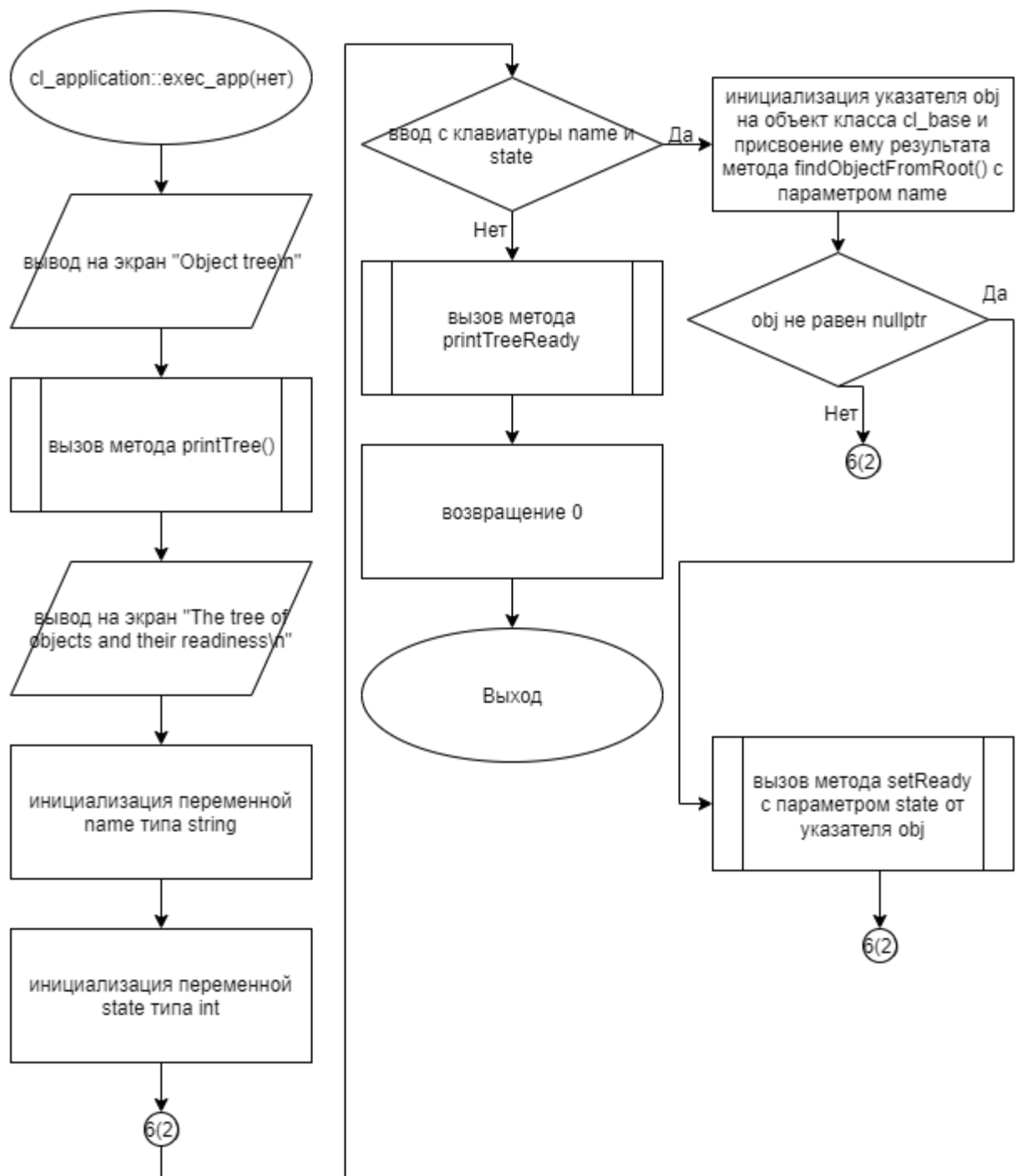


Рисунок 7 – Блок-схема алгоритма

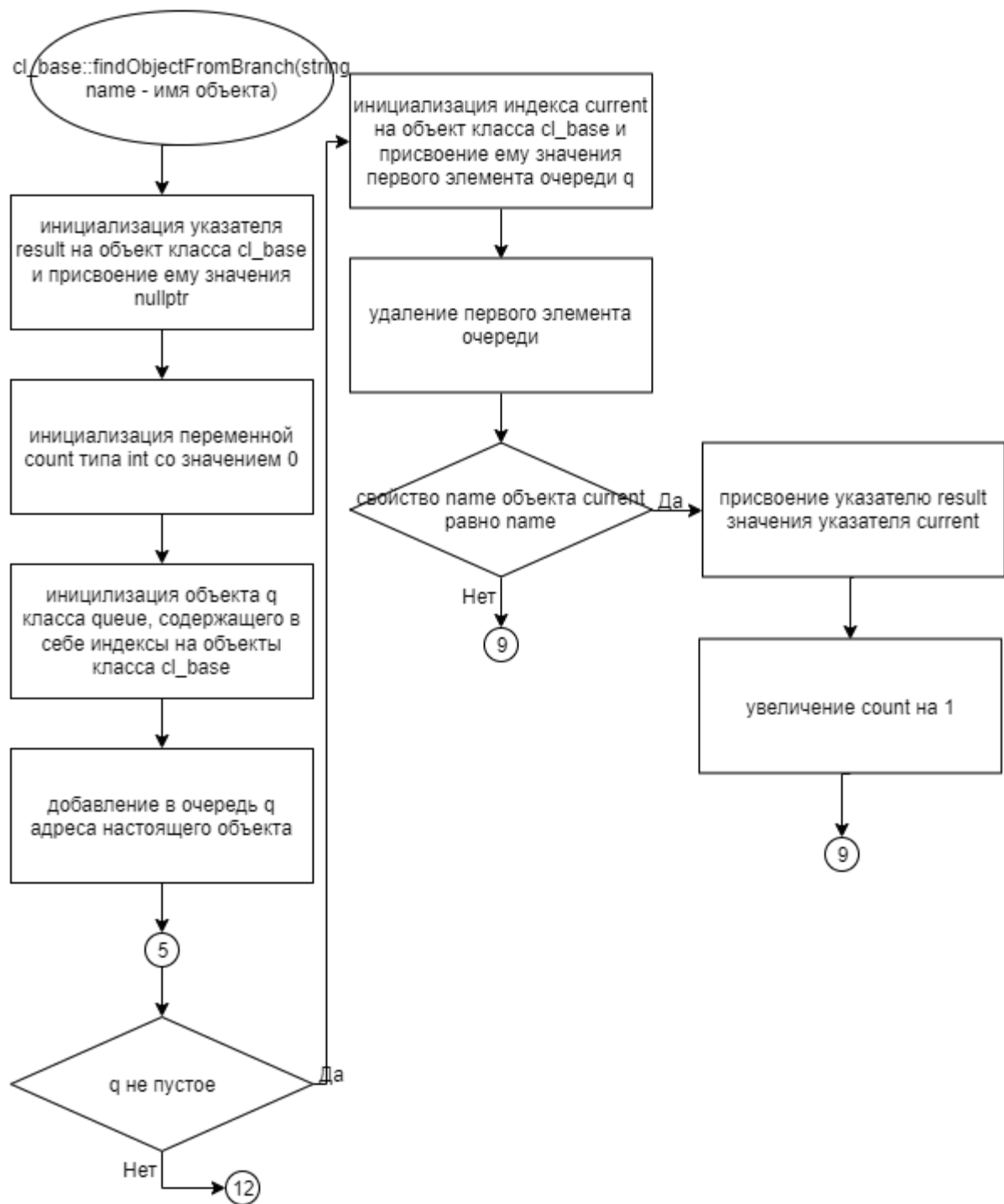


Рисунок 8 – Блок-схема алгоритма



Рисунок 9 – Блок-схема алгоритма

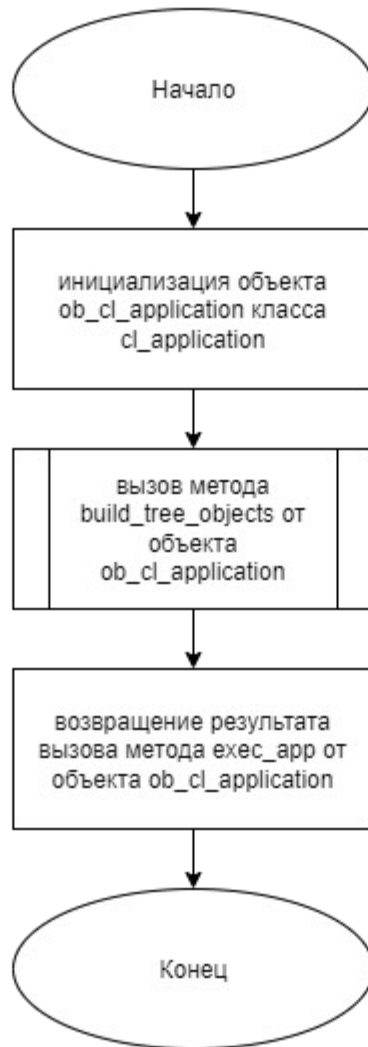


Рисунок 10 – Блок-схема алгоритма

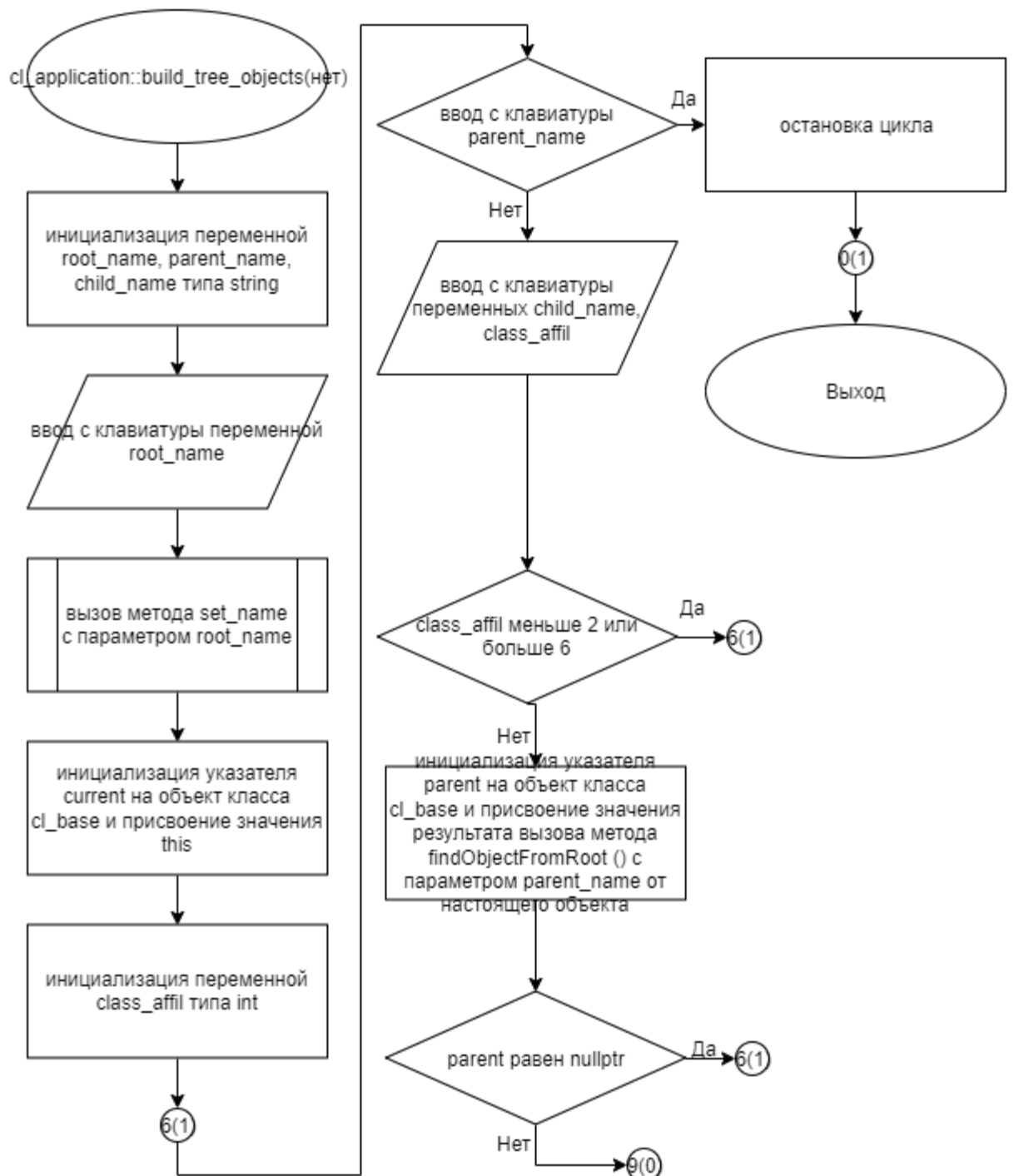


Рисунок 11 – Блок-схема алгоритма

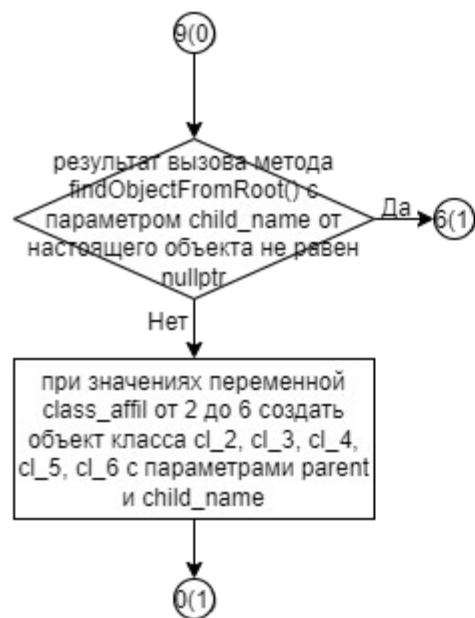


Рисунок 12 – Блок-схема алгоритма

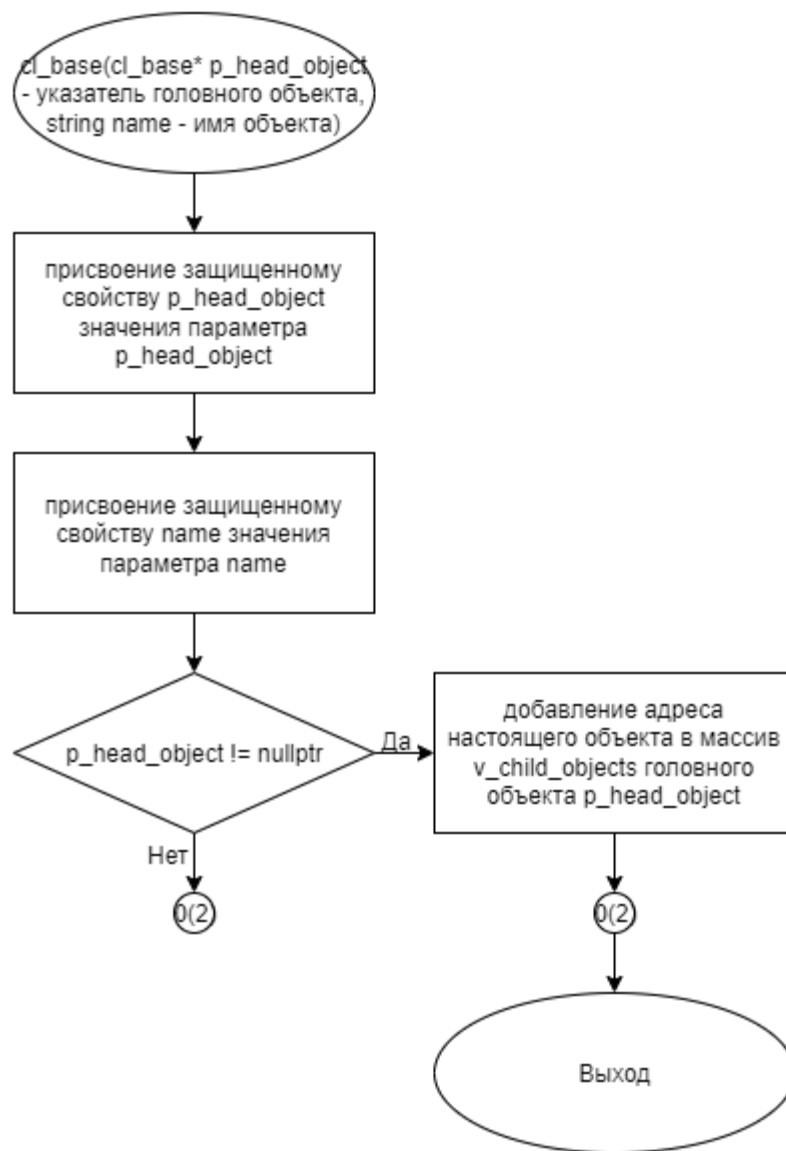


Рисунок 13 – Блок-схема алгоритма

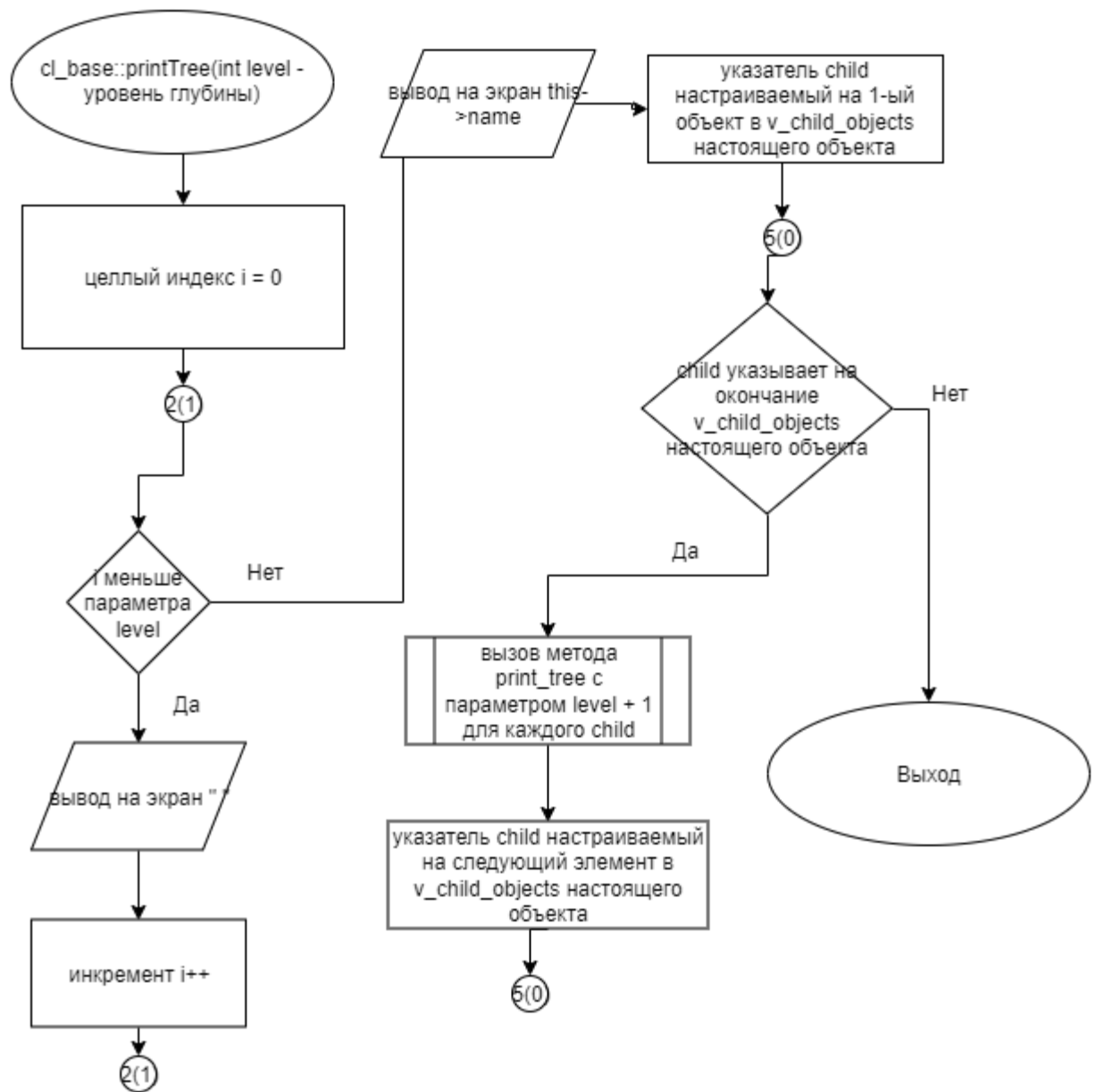


Рисунок 14 – Блок-схема алгоритма

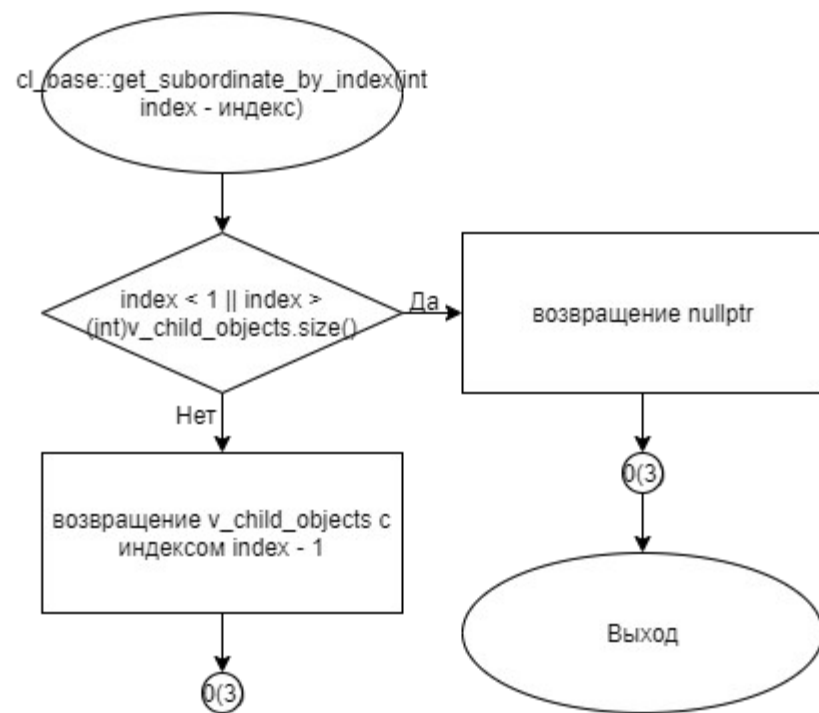


Рисунок 15 – Блок-схема алгоритма

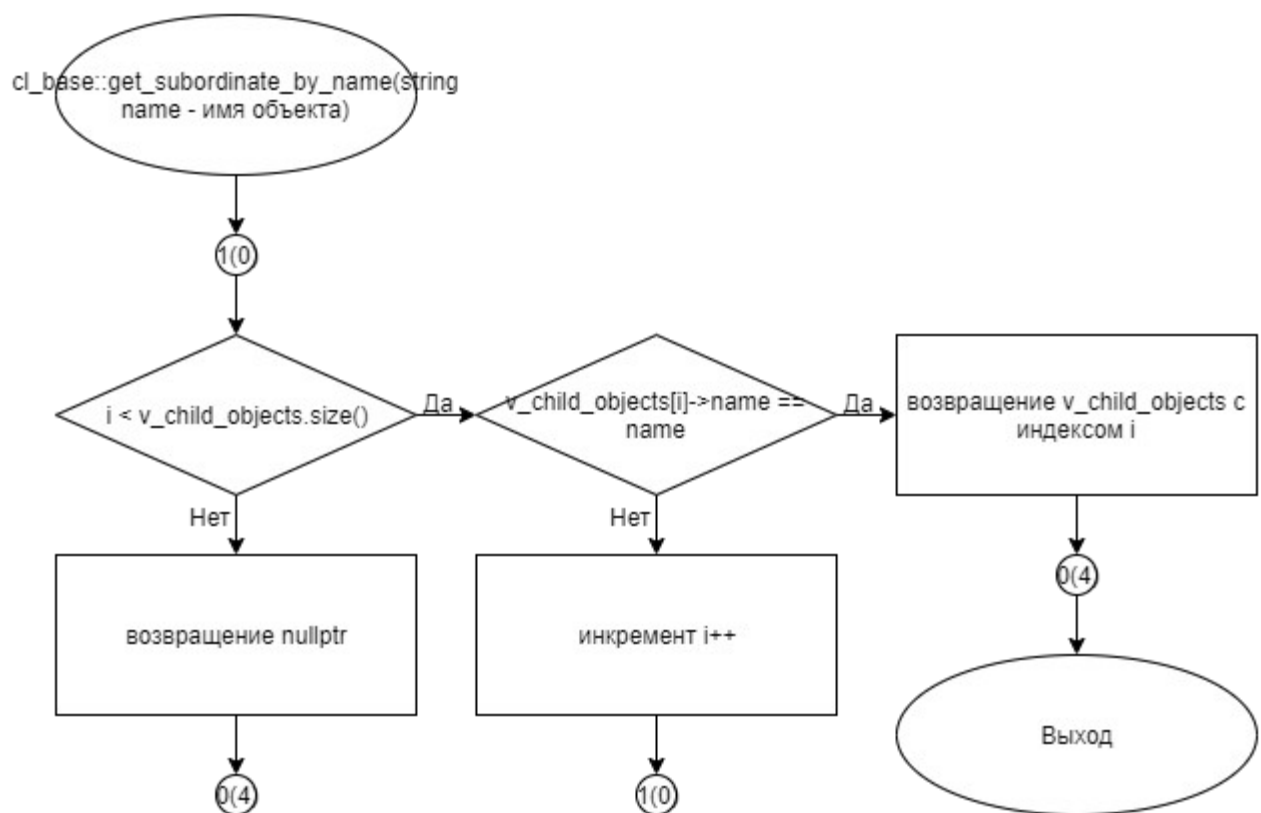


Рисунок 16 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл cl_2.cpp

Листинг 1 – cl_2.cpp

```
#include "cl_2.h"
#include <string>

cl_2::cl_2(cl_base* p_head_object, std::string name): cl_base(p_head_object,
name)
{
}
```

5.2 Файл cl_2.h

Листинг 2 – cl_2.h

```
#ifndef __CL_2__H
#define __CL_2__H

#include "cl_base.h"
#include <string>

class cl_2 : public cl_base
{
public:
    cl_2(cl_base* p_head_object, std::string name);
};

#endif
```

5.3 Файл cl_3.cpp

Листинг 3 – cl_3.cpp

```
#include "cl_3.h"
#include <string>

cl_3::cl_3(cl_base* p_head_object, std::string name): cl_base(p_head_object,
name)
{
}
```

5.4 Файл cl_3.h

Листинг 4 – cl_3.h

```
#ifndef __CL_3__H
#define __CL_3__H

#include "cl_base.h"
#include <string>

class cl_3 : public cl_base
{
public:
    cl_3(cl_base* p_head_object, std::string name);
};

#endif
```

5.5 Файл cl_4.cpp

Листинг 5 – cl_4.cpp

```
#include "cl_4.h"
#include <string>

cl_4::cl_4(cl_base* p_head_object, std::string name): cl_base(p_head_object,
name)
{
}
```

5.6 Файл cl_4.h

Листинг 6 – cl_4.h

```
#ifndef __CL_4__H
#define __CL_4__H

#include "cl_base.h"
#include <string>

class cl_4 : public cl_base
{
public:
    cl_4(cl_base* p_head_object, std::string name);
};

#endif
```

5.7 Файл cl_5.cpp

Листинг 7 – cl_5.cpp

```
#include "cl_5.h"
#include <string>

cl_5::cl_5(cl_base* p_head_object, std::string name): cl_base(p_head_object,
name)
{
}
```

5.8 Файл cl_5.h

Листинг 8 – cl_5.h

```
#ifndef __CL_5__H
#define __CL_5__H

#include "cl_base.h"
#include <string>

class cl_5 : public cl_base
{
public:
```

```
        cl_5(cl_base* p_head_object, std::string name);  
    };  
  
#endif
```

5.9 Файл cl_6.cpp

Листинг 9 – cl_6.cpp

```
#include "cl_6.h"  
#include <string>  
  
cl_6::cl_6(cl_base* p_head_object, std::string name): cl_base(p_head_object,  
name)  
{  
}  
}
```

5.10 Файл cl_6.h

Листинг 10 – cl_6.h

```
#ifndef __CL_6__H  
#define __CL_6__H  
  
#include "cl_base.h"  
#include <string>  
  
class cl_6 : public cl_base  
{  
public:  
    cl_6(cl_base* p_head_object, std::string name);  
};  
  
#endif
```

5.11 Файл cl_application.cpp

Листинг 11 – cl_application.cpp

```
#include "cl_application.h"
```

```

#include "cl_2.h"
#include "cl_3.h"
#include "cl_4.h"
#include "cl_5.h"
#include "cl_6.h"
#include <iostream>
#include <queue>
#include <vector>
#include <string>

cl_application::cl_application(cl_base* p_head_object):
cl_base(p_head_object)
{
}

void cl_application::build_tree_objects()
{
    std::string root_name, parent_name, child_name;
    std::cin >> root_name;
    set_name(root_name);

    cl_base* current = this;

    int class_affil;

    while (std::cin >> parent_name)
    {
        // проверка остановки
        if (parent_name == "endtree")
            break;

        std::cin >> child_name >> class_affil;

        // проверка границ класса
        if (class_affil < 2 || class_affil > 6)
            continue;

        cl_base* parent = this->findObjectFromRoot(parent_name);

        if (parent == nullptr)
            continue;

        // если такой объект уже есть
        if (this->findObjectFromRoot(child_name) != nullptr)
            continue;

        switch (class_affil)
        {
            case 2:
                current = new cl_2(parent, child_name);
                break;
            case 3:
                current = new cl_3(parent, child_name);
                break;

```

```

        case 4:
            current = new cl_4(parent, child_name);
            break;
        case 5:
            current = new cl_5(parent, child_name);
            break;
        case 6:
            current = new cl_6(parent, child_name);
            break;
    }
}

int cl_application::exec_app()
{
    std::cout << "Object tree\n";
    printTree();
    std::cout << "The tree of objects and their readiness\n";

    std::string name;
    int state;

    while (std::cin >> name >> state)
    {
        cl_base* obj = findObjectFromRoot(name);

        if (obj != nullptr)
        {
            obj->setReady(state);
        }
    }
    printTreeReady();

    return 0;
}

```

5.12 Файл cl_application.h

Листинг 12 – cl_application.h

```

#ifndef __CL_APPLICATION__H
#define __CL_APPLICATION__H

#include "cl_base.h"

class cl_application : public cl_base
{
public:
    cl_application(cl_base* p_head_object);

```

```

        void build_tree_objects();
        int exec_app();
    };

#endif

```

5.13 Файл cl_base.cpp

Листинг 13 – cl_base.cpp

```

#include "cl_base.h"
#include <string>
#include <queue>
#include <iostream>

cl_base::cl_base(cl_base* p_head_object, std::string name)
{
    this->p_head_object = p_head_object;
    this->name = name;

    if (p_head_object != nullptr)
    {
        p_head_object->v_child_objects.push_back(this);
    }
}

bool cl_base::set_name(std::string new_name)
{
    if (p_head_object == nullptr)
    {
        name = new_name;
        return true;
    }

    for (int i = 0; i < p_head_object->v_child_objects.size(); i++)
    {
        if (p_head_object->v_child_objects[i]->name == new_name)
        {
            return false;
        }
    }

    name = new_name;

    return true;
}

std::string cl_base::get_name()

```

```

{
    return name;
}

cl_base* cl_base::get_head_object()
{
    return p_head_object;
}

int cl_base::get_subordinate_count()
{
    return v_child_objects.size();
}

cl_base* cl_base::get_subordinate_by_index(int index)
{
    if (index < 1 || index > (int)v_child_objects.size())
    {
        return nullptr;
    }

    return v_child_objects[index - 1];
}

cl_base* cl_base::get_subordinate_by_name(std::string name)
{
    for (int i = 0; i < v_child_objects.size(); i++)
    {
        if (v_child_objects[i]->name == name)
        {
            return v_child_objects[i];
        }
    }

    return nullptr;
}

void cl_base::setReady(int state)
{
    if (state == 0)
    {
        is_ready = 0;

        for (auto child : this->v_child_objects)
        {
            child->setReady(0);
        }
        return;
    }

    // проверка родителей на наличие 0 в state
    cl_base* parent = p_head_object;

    while (parent != nullptr)
    {

```



```

        if (parent->is_ready == 0)
            return;

        parent = parent->p_head_object;
    }

    is_ready = state;
}

cl_base* cl_base::findObjectFromBranch(std::string name)
{
    cl_base* result = nullptr;
    int count = 0;

    std::queue<cl_base*> q;
    q.push(this);

    while (!q.empty())
    {
        cl_base* current = q.front();
        q.pop();

        if (current->name == name)
        {
            result = current;
            count++;
        }

        for (auto child : current->v_child_objects)
        {
            q.push(child);
        }
    }

    if (count == 1)
        return result;

    return nullptr;
}

cl_base* cl_base::findObjectFromRoot(std::string name)
{
    // находим корень
    cl_base* root = this;

    while (root->p_head_object != nullptr)
    {
        root = root->p_head_object;
    }

    return root->findObjectFromBranch(name);
}

void cl_base::printTree(int level)
{ // в данном случае level - уровень вложенности

```

```

        for (int i = 0; i < level; i++)
        {
            std::cout << "    ";
        }

        std::cout << this->name << std::endl;

        for (auto child : v_child_objects)
        {
            child->printTree(level + 1);
        }
    }

void cl_base::printTreeReady(int level)
{
    for (int i = 0; i < level; i++)
    {
        std::cout << "    ";
    }

    std::cout << name;

    if (is_ready != 0)
    {
        std::cout << " is ready" << std::endl;
    }
    else
    {
        std::cout << " is not ready" << std::endl;
    }

    for (auto child : v_child_objects)
    {
        child->printTreeReady(level + 1);
    }
}

cl_base::~cl_base()
{
    for (int i = 0; i < v_child_objects.size(); i++)
    {
        delete v_child_objects[i];
    }
}

```

5.14 Файл cl_base.h

Листинг 14 – cl_base.h

```
#ifndef __CL_BASE__H
```

```

#define __CL_BASE__H

#include <string>
#include <vector>

class cl_base
{
protected:
    std::string name;
    cl_base *p_head_object;
    std::vector<cl_base*> v_child_objects;

    int is_ready = 0;

public:
    cl_base(cl_base* p_head_object, std::string name = "Base_object");
    bool set_name(std::string new_name);
    std::string get_name();

    cl_base* get_head_object();

    int get_subordinate_count();

    // установка state
    void setReady(int is_ready);
    // вывод дерева
    void printTree(int level = 0);
    // вывод дерева + готовность
    void printTreeReady(int level = 0);

    // получить адрес по индексу
    cl_base* get_subordinate_by_index(int index);
    // получить адрес по имени
    cl_base* get_subordinate_by_name(std::string name);
    // найти адрес объекта по имени от настоящего объекта
    cl_base* findObjectFromBranch(std::string name);
    // найти адрес объекта по имени по всему дереву
    cl_base* findObjectFromRoot(std::string name);
    ~cl_base();
};

#endif

```

5.15 Файл main.cpp

Листинг 15 – main.cpp

```

#include <stdlib.h>
#include <stdio.h>
#include "cl_application.h"

```

```
int main()
{
    cl_application ob_cl_application(nullptr);
    ob_cl_application.build_tree_objects();
    return ob_cl_application.exec_app();
}
```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 19.

Таблица 19 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
app_root app_root object_01 3 app_root object_02 2 object_02 object_04 3 object_02 object_05 5 object_01 object_07 2 endtree app_root 1 object_07 3 object_01 1 object_02 -2 object_04 1	Object tree app_root object_01 object_07 object_02 object_04 object_05 The tree of objects and their readiness app_root is ready object_01 is ready object_07 is not ready object_02 is ready object_04 is ready object_05 is not ready	Object tree app_root object_01 object_07 object_02 object_04 object_05 The tree of objects and their readiness app_root is ready object_01 is ready object_07 is not ready object_02 is ready object_04 is ready object_05 is not ready
app_root app_root obj_1 1 app_root obj_2 2 app_root obj_3 3 app_root obj_4 10 obj_2 obj_3 4 obj_2 obj_5 4 endtree app_root -100 obj_2 5 obj_5 6	Object tree app_root obj_2 obj_5 obj_3 The tree of objects and their readiness app_root is ready obj_2 is ready obj_5 is ready obj_3 is not ready	Object tree app_root obj_2 obj_5 obj_3 The tree of objects and their readiness app_root is ready obj_2 is ready obj_5 is ready obj_3 is not ready
app_root app_root obj_1 1 app_root obj_2 2 app_root obj_3 3 obj_2 obj_5 4 obj_10 obj_8 2 obj_5 obj_5 5 endtree app_root 1	Object tree app_root obj_2 obj_5 obj_3 The tree of objects and their readiness app_root is not ready obj_2 is not	Object tree app_root obj_2 obj_5 obj_3 The tree of objects and their readiness app_root is not ready obj_2 is not

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
obj_2 2 obj_3 3 obj_5 5 app_root 0	ready obj_5 is not ready obj_3 is not ready	ready obj_5 is not ready obj_3 is not ready

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.
2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avvora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).
3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avvora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).
4. Шилдт Г. С++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.
5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».
6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).